

Mycelium 4.0

Rapport de conception

ALLAY Mohamed (actuellement en mobilité) - BOTTOIS Elise - DUFOURCQ Thibault -
LAHMAMSI Mohamed - LÉCRIVAIN Arno

Encadrants : PARLAVANTZAS Nikos - PAROL-GUARINO Volodia

Avec la participation de : MOUREAU Julien

2024/2025



Contents

1	Introduction	3
2	Architecture du projet	4
2.1	Architecture matérielle	4
2.1.1	Noeuds SoLo	5
2.1.2	Cluster de Raspberry PI	5
2.1.3	VPS de l'INSA	6
2.2	Architecture Logicielle	6
2.2.1	Kubernetes : exécution conteneurisée sur un <i>cluster</i> d'ordinateurs	7
2.2.2	OpenFaas et MQTT : gestion des fonctions de traitement des données	7
2.2.3	ChirpStack : gestion de la communication LoRaWAN	7
2.2.4	Composants logiciels de périphérie	8
2.2.5	Machine virtuelle de développement	9
3	Ajouts de Mycélium 4.0	9
3.1	Scénario Changement de fréquence	9
3.1.1	Architecture du scénario	10
3.1.2	Acquisition des Données	10
3.1.3	Stockage des Données dans InfluxDB	10
3.1.4	Analyse Statistique et Détection des Événements Météorologiques	11
3.1.5	Détection des Seuils Critiques	12
3.1.6	Modélisation des Variations Rapides	12
3.1.7	Implémentation Technique de l'Adaptation	12
3.1.8	Intégration d'un modèle de régression multiple pour la prédiction de la fréquence	14
3.2	Implémentation d'un proxy au sein de l'architecture	14
3.2.1	Présentation Générale du Proxy	15
3.2.2	Exemple d'Exécution du Proxy	17
3.2.3	Monitoring du Cluster	17
3.2.4	Politique de Routage des Messages MQTT	17
3.2.5	Avantages de l'Architecture	18
3.3	Intégration du scénario inondation	18
3.3.1	Adaptation à l'architecture Mycélium	18
3.3.2	Ajustement du modèle prédictif	18
4	Conclusion	19

1 Introduction

Mycélium se place dans le cadre du projet TERRAFORMA d’une échelle nationale qui s’inscrit dans une démarche de développement d’observations et d’expérimentations socio-écologiques des territoires pour tendre vers des transformations durables et équitables. Cette initiative part d’un constat de nécessité de préservation de l’habitabilité de la planète et passe par la mise en place de sites témoins tels que le site de la Croix Verte (figure 1) dans le cadre de Mycélium. En effet, ce site est pertinent à suivre suite à l’impact écologique qu’a eu la construction de la ligne de métro sur Beaulieu à Rennes.

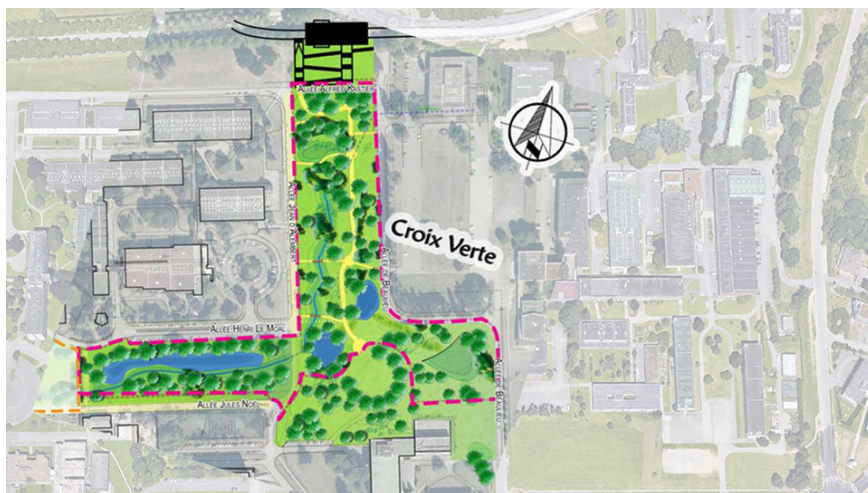


Figure 1: Représentation cartographique de la zone de la Croix Verte sur le campus de Beaulieu

Au-delà des objectifs environnementaux-sociaux-économiques précédemment cités, l’intérêt particulier de Mycélium réside dans les technologies mises en place pour ce suivi environnemental. Celles-ci permettent en effet de s’affranchir des manipulations manuelles à l’aide du déploiement d’un réseau de capteurs intelligents. C’est là que le projet Mycélium intervient pour mettre en place cette innovation à l’échelle du site de Beaulieu. Dans le cas de Mycélium, l’utilisateur principal visé est pour l’instant notre partenaire, l’OSUR, dont les ingénieurs collectent et exploitent les données, mais l’objectif à long terme qui meut Terra Forma est de rendre ces données OpenSource et accessibles aux acteurs locaux pour qu’ils s’en emparent et agissent collectivement dans cet objectif de transformations équitables et durables. L’utilisateur peut donc être un scientifique chargé de la surveillance de la zone de la Croix Verte. En prenant l’exemple du scénario inondation, si un niveau d’eau critique est détecté par les capteurs, l’utilisateur final sera alors notifié.

Les versions 1.0 et 2.0 de Mycélium ont posé les bases de l’infrastructure encore d’actualité malgré des évolutions avec le développement d’abord d’un nœud de capteurs basse énergie, l’utilisation du protocole LoRaWAN et l’architecture de Fog Computing. Mycélium 3.0 fut une refonte globale du projet visant à optimiser et pérenniser sa manipulation, avec la création d’un guide, la mise en place d’une machine virtuelle qui émule le comportement d’un cluster. Mycélium 4.0 quant à lui tente de mettre en place une base de développement plus simple et d’intégrer de nouveaux scénarios utiles et résilients aux déconnexions, avec l’implémentation d’un proxy. Un autre objectif est d’intégrer un retour d’information pour contrôler les capteurs automatiquement.

Ce rapport de conception vise à détailler l’architecture interne du projet déjà évoquée dans le précédent rapport de spécification. Le fonctionnement interne de chaque module sera cette fois davantage détaillé, ainsi que l’articulation entre ceux-ci. L’architecture fonctionnelle actuelle du projet selon ses versions précédentes sera d’abord présentée, dans la section 2, avant d’exposer dans un second temps l’architecture et le fonctionnement global du projet Mycélium 4.0 dans la section 3. Pour bien comprendre ces nouvelles intégrations, le fonctionnement global de nos technologies sera exposé avec l’exemple des fonctions inondation, puis les

fonctions de changement de fréquence seront décrites, ainsi que le fonctionnement du proxy, composant apparenté à un serveur relais qui dans notre cas fait l'intermédiaire entre le cluster et le VPS (Virtual Private Server).

2 Architecture du projet

L'architecture du projet Mycelium évolue au fil de ses versions, tant sur le plan matériel que logiciel. Pour expliquer nos ajouts dans la version 4.0, nous allons d'abord devoir présenter l'architecture actuelle. Cette architecture reste inchangée au niveau matériel (par rapport à la version 3.0), mais présente des améliorations dans sa structure logicielle.

2.1 Architecture matérielle

L'architecture matérielle du projet (représentée dans la figure ci-dessous) est donc identique à celle de la version 3.0, et distingue trois sous-ensembles principaux : les nœuds SoLo, le *cluster* Raspberry, et le VPS INSA. L'architecture est aussi agrémentée de plusieurs dispositifs permettant l'interaction et l'exploitation de ces composants.

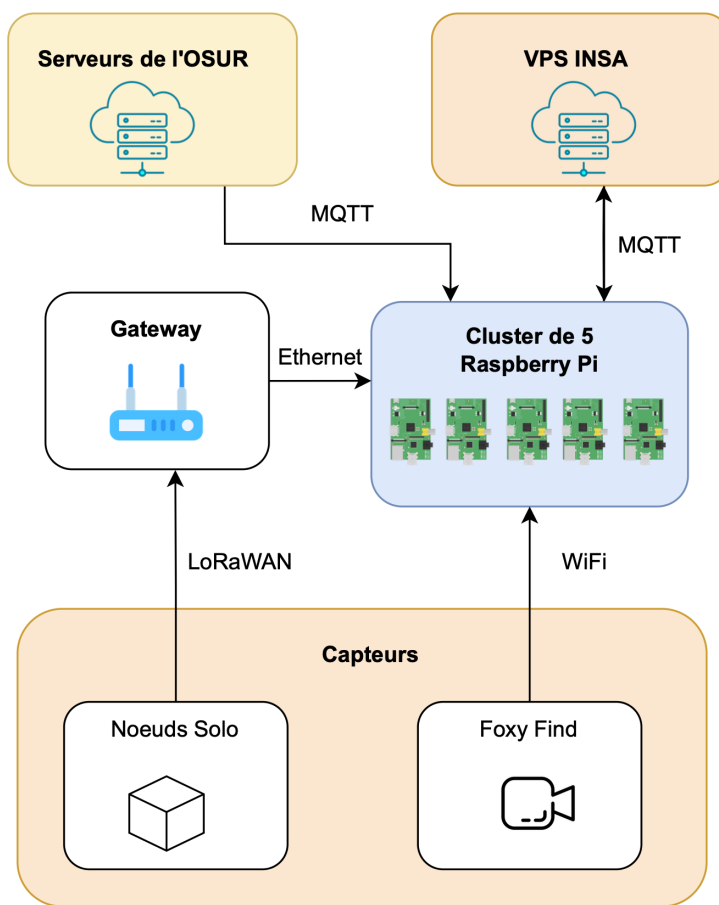


Figure 2: Schéma de l'architecture matérielle

2.1.1 Noeuds SoLo

Nous comptons d’abord le système principal d’acquisition de données, soit l’ensemble des assemblages de capteurs sur site (Croix-Verte), aussi appelés noeuds Solo [13]. Chaque noeud SoLo est contenu dans un boîtier étanche qui abrite plusieurs capteurs (de température, d’humidité, de luminosité, d’accéléromètre, de pression, et de pluviomètre).

Ces capteurs récoltent diverses données environnementales de manière autonome, puis les transmettent à l’aide du protocole radio de télécommunication LoRaWAN (Long Range Wide Area Network). Ce protocole permet la transmission de petits paquets de données en bas débit, sur une fréquence de 868 MHz. Il assure une transmission fiable d’une faible quantité de données, sur une portée de plusieurs kilomètres, tout en garantissant une consommation énergétique très faible. Il est donc particulièrement adapté aux objets connectés, et par extension, à l’architecture de notre projet.

Les capteurs de chaque noeud SoLo sont paramétrables à l’aide de fichiers au format JSON. Cela permet de configurer dynamiquement (pendant le fonctionnement) leur comportement, comme par exemple la fréquence d’envoi des données, la fréquence de prise de mesures, le réseau, etc. . . Cette adaptabilité sera appréciée pour augmenter la précision à la détection d’évènements importants (risque d’inondation, suspicion de vol de capteur, . . .), et à l’inverse économiser de l’énergie et des ressources le reste du temps.

Cette autonomie de traitement des capteurs entrent en résonance avec le choix d’un système fog pour notre architecture. Le fog computing [16] est une architecture (extension du cloud computing [17]) qui traite et stocke les données plus près des appareils et des utilisateurs, souvent à la périphérie du réseau, pour réduire la latence et améliorer l’efficacité des services connectés.

En plus de ces noeuds SoLo, l’OSUR nous donne accès à ses capteurs répartis sur le campus (qui sont aussi des noeuds SoLo). La stratégie de communication choisie pour récupérer leurs données n’est pas exactement celle de nos capteurs (expliquée en section 2.2.4). Il existe également un appareil Raspberry Pi (hors *cluster*) appelé Foxy Find. Il a pour but de détecter le passage d’un renard, et utilise une caméra qui s’ajoute à notre batterie de capteurs. Cette dernière est connectée au reste de Mycélium via WiFi.

Voyons maintenant comment ces données sont traitées.

2.1.2 Cluster de Raspberry PI

Le *cluster* de Raspberry Pi, situé au sein du bâtiment INFO, est le deuxième sous-ensemble principal de l’architecture 3.0. C’est un assemblage de 5 cartes Raspberry Pi [20] de 1 Go de RAM chacune (ordinateur peu coûteux de la taille d’une carte de crédit).

Une des Raspberry est considérée comme un noeud superviseur. Les quatre autres sont des noeuds contrôleurs, recevant les ordres du noeud superviseur. Cela forme un *cluster* qui aura pour objectif d’effectuer tous les traitements et analyses sur les données récoltées par les capteurs. Ce *cluster* est l’élément central de l’architecture, qui effectue la plupart des opérations, et ce en continu.

Le *cluster* est connecté à une *gateway* LoRaWAN. La *gateway* agit comme une passerelle de communication entre les noeuds SoLo et un réseau IP. Elle centralise les données provenant des capteurs, les recevant depuis les noeuds SoLo et les renvoyant via une connexion Ethernet vers le superviseur du *cluster* de Raspberry Pi.

Les paquets de données provenant des capteurs sont réceptionnés, décodés, répartis, puis traités sur le *cluster*. Cependant, l’une des contraintes handicapantes inhérentes à ce système fog est le peu de mémoire vive disponible. En effet, la charge de calcul peut parfois dépasser les capacités du *cluster*, et nécessite donc d’être déléguée . . .

2.1.3 VPS de l'INSA

Dans le cas où les ressources du *cluster* viendraient à manquer, un VPS (Virtual Private Server) a été ajouté. Ce serveur est le troisième sous-ensemble principal de l'architecture 3.0. Il est hébergé par l'INSA, et permet de venir en aide au *cluster*. Ainsi, au-delà d'un certain palier d'utilisation du *cluster* (précisé en section 3.2.3), les calculs sont effectués sur le serveur de l'INSA.

Ce VPS présente aussi d'autres fonctionnalités dans l'architecture. Il possède une fonction de stockage postérieure au *cluster*, nous permettant de générer des résumés journaliers des acquisitions du *cluster*. De plus, il hébergera l'interface utilisateur que consultera principalement le client. En effet, c'est sur le cloud de l'INSA que l'utilisateur final se connectera pour visualiser les données ainsi que les notifications (abordées en section 2.2.4).

Nous avons vu quels étaient les composants matériels de l'architecture actuelle, leur rôle, et leurs interactions. Regardons maintenant comment ils fonctionnent d'un point de vue logiciel.

2.2 Architecture Logicielle

L'architecture logicielle a quant à elle évolué. Nous allons donc présenter l'architecture actuelle (représentée dans la figure ci-dessous), en expliquant les concepts et technologies issues de la version 3.0, dans cette section. Nous aborderons ensuite les ajouts et nouveautés de la version 4.0 dans la section suivante (section 3).

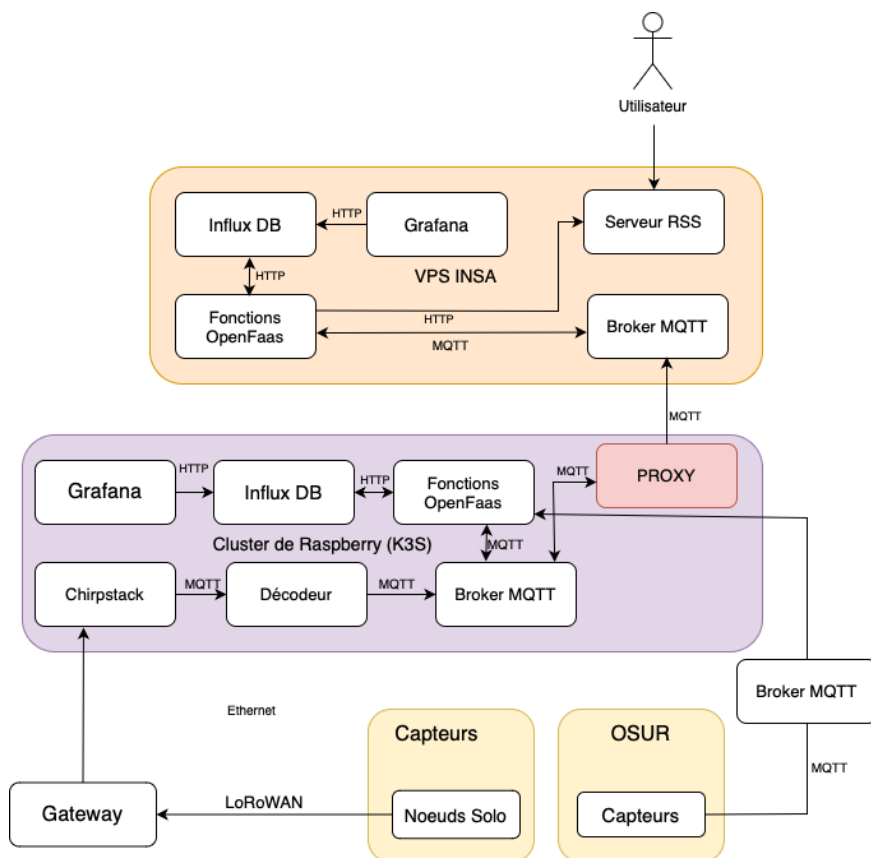


Figure 3: Schéma de l'architecture logicielle

Pour ce faire, commençons par expliquer comment les modules Raspberry Pi du *cluster* fonctionnent ensemble, et nous permettent d'effectuer les différents traitements des données, de manière asynchrone.

2.2.1 Kubernetes : exécution conteneurisée sur un *cluster* d'ordinateurs

Kubernetes (K8S) [11] est une plateforme open source destinée à automatiser le déploiement, la mise à l'échelle et la gestion des applications conteneurisées. Il facilite la gestion d'applications distribuées en microservices sur un *cluster* de machines. Cette plateforme nous permet d'orchestrer efficacement les ressources de nos différents modules RaspBerry Pi, et son appétence pour l'organisation en microservices rejoint notre besoin de traitement parallèle et asynchrone des données. La conteneurisation facilite aussi le déploiement des différentes applications utilisées à chaque étape de la chaîne de traitement.

Pour notre *cluster*, nous choisirons d'utiliser (sur un linux) K3S [21], une distribution légère de Kubernetes conçue pour les environnements avec des ressources limitées, tels que les systèmes embarqués, les dispositifs IoT ou les clusters de développement. Cette distribution correspond donc parfaitement à notre utilisation.

2.2.2 OpenFaaS et MQTT : gestion des fonctions de traitement des données

Les données des capteurs sont réceptionnées à des intervalles de temps non-prévisibles. Pour assurer le traitement asynchrone de ces données, nous utilisons une organisation du code en fonctions, selon le principe de Function as a Service (FaaS) [22]. En d'autres termes, chaque service est une fonction qui remplit une unique tâche. Ces fonctions peuvent en appeler d'autres à leur terminaison, et ainsi construire une chaîne de traitements.

OpenFaaS [9] sera utilisé afin d'implémenter cette architecture. C'est une plateforme open-source qui facilite le déploiement et la gestion de fonctions serverless [10], permettant ainsi l'exécution de code de manière événementielle sans se soucier de l'infrastructure sous-jacente.

Les fonctions se servent d'un *event broker* [23] pour communiquer. En effet, la gestion des appels aux fonctions de traitement est effectuée par un *broker* MQTT (Mosquitto dans notre cas [8]) : un serveur intermédiaire qui facilite la communication asynchrone entre les dispositifs IoT, en gérant les opérations de *publish* (publication) et de *subscribe* (abonnement) pour l'échange de messages. MQTT (Message Queuing Telemetry Transport) [6], est un protocole de messagerie léger *publish-subscribe* basé sur le protocole TCP/IP.

Ainsi, pour appeler une fonction (suite à une réception de données, ou un appel d'une autre fonction), il faut publier sur un *topic* du *broker* [7] auquel s'est abonnée la fonction en question. Le *broker* MQTT est donc le composant central de cette architecture FaaS pour le traitement des données.

2.2.3 ChirpStack : gestion de la communication LoRaWAN

L'architecture logicielle du *cluster* intègre ChirpStack [24], un ensemble de logiciels open source permettant la gestion et la configuration d'appareils utilisant le protocole LoRaWAN. La *gateway* va réceptionner les transmissions LoRaWAN des capteurs, et les envoyer au *cluster* via ethernet. Dans le *cluster*, Chirpstack va réceptionner les données provenant de la Gateway, sous forme de paquets UDP, va ensuite les décoder, puis les transférer vers les fonctions de traitement du *cluster* à l'aide du *broker* MQTT.

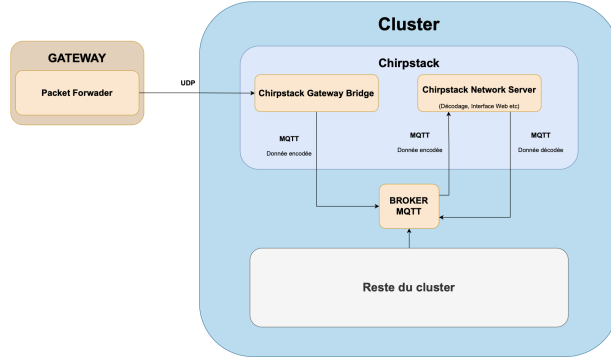


Figure 4: Schéma de l'architecture de ChirpStack

Chirpstack est structuré autour de plusieurs sous-composants, notamment le ChirpStack Gateway Bridge. Ce composant agit comme un intermédiaire entre la *gateway* LoRaWAN et un autre composant, le ChirpStack Network Server. Plus exactement, il reçoit les données encodées de la *gateway* LoRaWAN, les transforme en un format exploitable, puis les publie via le protocole MQTT sur un *topic* du *broker*, pour qu'elles soient utilisées par ChirpStack Network Server. ChirpStack Network Server permet de décoder les données à l'aide de scripts écrits en Python et JavaScript. Les données décodées sont ensuite publiées sur le *topic* MQTT de traitement correspondant. Ce composant permet aussi de gérer les capteurs et la *gateway*. Il propose l'ajout de nouveaux capteurs, ainsi que la configuration de la *gateway* et des capteurs, via une interface Web mise à disposition sur le port 8080.

2.2.4 Composants logiciels de périphérie

Les technologies que nous venons de voir permettent de réaliser le traitement ordinaire des données. Cependant, nous utilisons aussi d'autres composants logiciels pour gérer des cas plus exceptionnels, ou pour faciliter l'interaction en périphérie du réseau.

Nous noterons par exemple l'utilisation de Grafana [18] pour visualiser les données stockées en base. Aussi, les données issues des capteurs de l'OSUR entreront dans la chaîne de traitements directement au format MQTT, via le broker (après un passage dans le broker MQTT propre de l'OSUR). Ces données sont immédiatement remontées vers le VPS pour y être analysées ultérieurement par l'utilisateur, elles ne subissent pas de traitement au sein du *cluster*.

La communication entre le *cluster* et le VPS sera effectuée par un proxy (section 2.2.2). Comme dit plus haut (section 2.1.3), la même base architecturale (K3S, MQTT, OpenFaaS, influxDB, Grafana) est aussi déployé sur le VPS, pour assurer la continuité du traitement des données. La finalité de cette chaîne de traitement résultera en un stockage de données (sur *cluster* ou sur VPS), ou l'envoi de notifications. Ces notifications s'afficheront dans l'interface utilisateur qu'intégrera le VPS, via un serveur RSS [19].

Enfin, nous organisons le code des fonctions de traitement en scénarios. Les scénarios consistent en la détection d'un événement (via les données reçues), et la prise de mesures conséquentes (action sur les données reçues, reparamétrage des capteurs, envoi de notifications à l'utilisateur, ...). Certaines des fonctions FaaS servent donc à l'implémentation d'un ou plusieurs scénarios, représentant ainsi l'aspect applicatif du projet. Autrement dit, l'objectif final est d'écrire tout le code bas-niveau permettant l'implémentation générique de scénarios, pour que le cahier des charges des futures équipes de développement consiste uniquement en la recherche et l'implémentation de ces scénarios.

Nous noterons d'ailleurs que le passage à la version 4.0 a rendu obsolète les anciens scénarios. Les scénarios que nous tentons d'intégrer à notre nouvelle architecture seront expliqués dans la section suivante (section 3).

2.2.5 Machine virtuelle de développement

Le *cluster* de Raspberry Pi que nous utilisons pour traiter les données récoltées est au centre de notre projet, cependant il est difficile de travailler directement dessus pour plusieurs raisons. Premièrement parce qu'il nécessite un accès direct au matériel, donc d'être physiquement présent au bâtiment informatique de l'INSA Rennes, et d'avoir la clé de la salle où il est stocké. Mais aussi car cela crée un risque de retour en arrière de ses fonctionnalités, voir de le rendre inopérant, en cas de mauvaises manipulations.

Ainsi par souci de prévention et de flexibilité nous travaillons, avant d'implémenter des changements sur le *cluster*, sur une émulation, soit la VM (Virtual Machine) de développement. Pour cela, nous utilisons le logiciel de virtualisation open source QEMU (Quick EMUlator) [25].

L'an précédent, la machine virtuelle a facilité le développement, or seulement un élève l'a fait marcher sur la fin du projet puisqu'elle n'était pas reproductible ailleurs. C'est pourquoi nous avons décidé cette fois de mettre en place une VM reproductible basée sur Nix qui a été fournie par Mr. PAROL-GUARINO et modifiée par nos soins pour émuler un cluster kubernetes avec un broker mqtt, un flux RSS et une base de données InfluxDB.

3 Ajouts de Mycélium 4.0

Dans cette section, nous présentons les nouveaux ajouts apportés à Mycélium 4.0, visant à améliorer l'adaptabilité, la résilience et la pertinence du système. Ces évolutions permettent d'optimiser l'utilisation des ressources, d'améliorer la gestion des messages et d'intégrer de nouveaux scénarios applicatifs. Trois principales améliorations sont introduites :

- **Changement dynamique de fréquence des capteurs** (section 3) : une adaptation intelligente de l'échantillonnage en fonction des conditions environnementales pour optimiser la consommation énergétique et la pertinence des données collectées.
- **Implémentation d'un proxy** (section 3.2) : un composant clé pour orchestrer efficacement la distribution des messages entre le cluster et le VPS, garantissant un routage optimisé et une meilleure résilience du système.
- **Intégration d'un scénario d'inondation** (section 3.3) : un modèle prédictif basé sur des réseaux de neurones LSTM permettant d'anticiper les crues et d'alerter l'utilisateur via un flux RSS.

Ces ajouts viennent compléter les scénarios existants de Mycélium 3.0, qui incluaient déjà des mécanismes de surveillance environnementale et de gestion des capteurs sur un réseau LoRaWAN. Cette nouvelle version renforce ainsi l'efficacité et la robustesse du système face aux évolutions des besoins et aux défis environnementaux.

3.1 Scénario Changement de fréquence

L'adaptation dynamique de la fréquence d'échantillonnage des capteurs, en fonction des conditions environnementales et des événements détectés, joue un rôle important dans la réduction de la consommation énergétique et dans la maximisation de la pertinence des données collectées. En effet, un système de surveillance environnementale doit être capable de répondre en temps réel aux variations de son environnement tout en optimisant les ressources dont il dispose. Cette partie présente une architecture de système modulaire et flexible permettant la gestion dynamique de la fréquence des capteurs, en fonction des résultats d'analyses statistiques et des événements climatiques détectés auparavant. Nous détaillerons ici les différents composants du système, leur fonctionnement, ainsi que les algorithmes et méthodes statistiques utilisés pour assurer la fiabilité et l'efficacité du processus.

3.1.1 Architecture du scénario

Le scénario repose sur une architecture modulaire, dans laquelle chaque composant joue un rôle spécifique mais interconnecté. Les modules du système sont conçus pour collecter, stocker, analyser et ajuster les données et notre système en fonction des événements environnementaux. L'acquisition des données se fait par des capteurs, qui mesurent divers paramètres environnementaux tels que la température, l'humidité ou la pression. Ces données sont ensuite stockées dans une base de données temporelle, plus précisément InfluxDB, qui est utilisée pour stocker les séries temporelles collectées. Une analyse statistique est ensuite réalisée pour identifier les événements météorologiques anormaux. Sur la base des résultats obtenus, le système ajuste la fréquence d'échantillonnage des capteurs afin de répondre de manière optimale aux événements détectés. Le diagramme ci-dessous (Figure 5) explique le fonctionnement de ce scénario, et une explication détaillée de chaque fonction sera donnée par la suite :

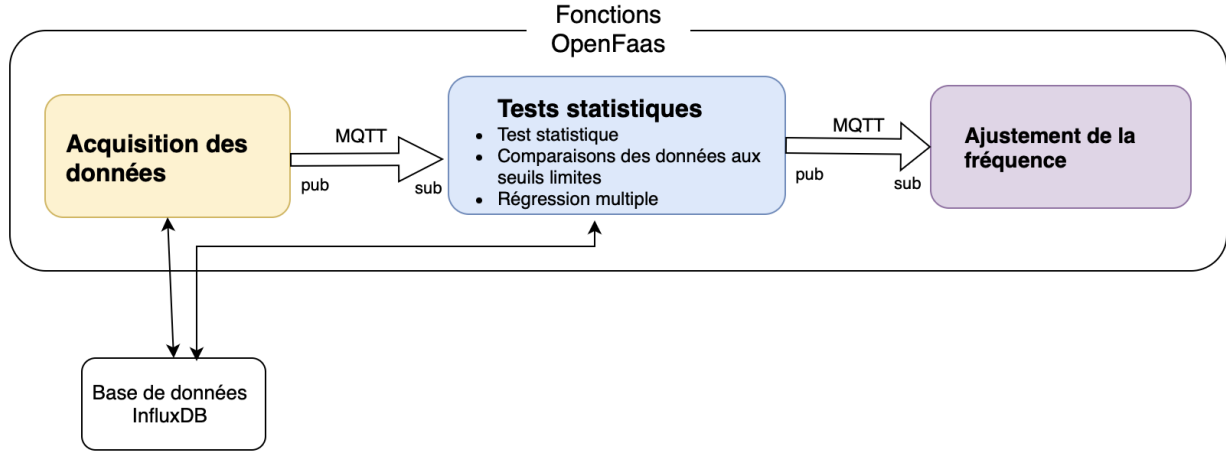


Figure 5: Fonctionnement des fonctions du scénario "Changement de fréquence".

3.1.2 Acquisition des Données

L'acquisition des données est réalisée à travers des capteurs environnementaux qui mesurent des paramètres tels que la température (T), l'humidité (H) et la pression (P). Ces capteurs sont conçus pour capturer des données à des intervalles réguliers, généralement de manière continue, et ces mesures sont ensuite envoyées à un module de stockage. L'un des enjeux majeurs ici est de déterminer à quelle fréquence les capteurs doivent envoyer les données, en fonction des conditions météorologiques détectées.

Les données sont envoyées sous forme de séries temporelles. Par exemple, pour la température, la mesure peut être notée sous la forme suivante :

$$T(t) = \{(t_0, T_0), (t_1, T_1), (t_2, T_2), \dots, (t_n, T_n)\}$$

où t_i représente un timestamp et T_i la température mesurée à ce moment-là.

3.1.3 Stockage des Données dans InfluxDB

InfluxDB est une base de données conçue spécifiquement pour le stockage et l'analyse de séries temporelles, c'est-à-dire des données horodatées générées en continu. Contrairement aux bases relationnelles classiques, elle est optimisée pour gérer efficacement de grands volumes de données chronologiques avec une faible latence, ce qui la rend particulièrement adaptée aux applications de surveillance environnementale, d'IoT ou encore d'analyse de performances.

Elle repose sur trois concepts clés :

- **Measurement** : Équivalent à une table en base relationnelle, il regroupe les données d'un capteur. Exemple : "environment data" pour stocker température, humidité, etc.
- **Fields** : Données mesurées (ex. temperature: 22.5, humidity: 55), non indexées mais utilisées pour les analyses.
- **Tags** : Métadonnées indexées pour filtrer efficacement les requêtes (ex. sensor_id: "sensor_1", location: "region_A").

Exemple d'enregistrement :

```
measurement: "environment_data"
tags: { "sensor_id": "sensor_1", "location": "region_A" }
fields: { "temperature": 22.5, "humidity": 55 }
timestamp: 2025-02-06T12:00:00Z
```

Ce modèle assure un stockage structuré, rapide et performant pour l'analyse des données environnementales.

3.1.4 Analyse Statistique et Détection des Événements Météorologiques

L'analyse statistique constitue le cœur de la gestion dynamique de la fréquence d'échantillonnage des capteurs. Cette analyse a pour objectif de détecter des événements météorologiques significatifs, tels que des vagues de chaleur ou des tempêtes, en fonction des séries temporelles de données collectées. Ces événements, lorsqu'ils sont détectés, entraînent un ajustement de la fréquence d'échantillonnage des capteurs pour augmenter la résolution des données collectées pendant des périodes critiques.

Pour identifier des événements météorologiques inhabituels, des tests statistiques sont appliqués aux données collectées afin de repérer des changements significatifs dans leur distribution ou leur variance. Ces analyses permettent d'ajuster la fréquence des mesures et d'anticiper d'éventuelles évolutions climatiques.

- **Test de Kolmogorov-Smirnov (KS) :**

Le test de Kolmogorov-Smirnov [1] compare la distribution des nouvelles données avec celle des données historiques pour vérifier s'il y a une différence significative. Il mesure la plus grande différence entre les fonctions de répartition empirique des deux échantillons. Si cette différence dépasse un seuil critique, cela signifie que les conditions météorologiques ont évolué de manière significative.

Dans le cadre de la surveillance environnementale, le test Kolmogorov-Smirnov permet de détecter des variations soudaines de température, d'humidité ou de pression atmosphérique après leur survenue. Bien qu'il facilite l'adaptation des modèles de prévision et l'ajustement des stratégies de mesure, son principal inconvénient est qu'il intervient a posteriori, ce qui peut entraîner un retard dans la réaction aux événements.

- **Test de Levene :**

Le test de Levene [2] évalue la stabilité de la variance des données. Une augmentation soudaine de la variance des valeurs peut indiquer une instabilité météorologique accrue, nécessitant une révision de la fréquence d'échantillonnage.

Ce test compare la variance de plusieurs groupes de données et détecte si elles diffèrent de manière significative. Une forte augmentation de la variance peut signaler un changement climatique brutal, comme l'arrivée d'une tempête ou une transition vers un nouveau régime météorologique.

En combinant ces deux tests, il devient possible de repérer rapidement des anomalies climatiques et d'optimiser la collecte des données pour améliorer la précision des prévisions météorologiques.

3.1.5 Détection des Seuils Critiques

En plus de ces tests statistiques, chaque paramètre mesuré, qu'il s'agisse de la température, de l'humidité ou de la pression, est comparé à des seuils critiques définis à partir de scénarios historiques. Par exemple, si la température dépasse un seuil défini comme une canicule, une alerte sera générée, et le système ajustera automatiquement la fréquence d'échantillonnage pour capturer plus de données à haute résolution pendant cette période.

Les seuils peuvent être définis par des intervalles spécifiques, par exemple:

$$T_{\text{critique}} = 35^{\circ}\text{C}$$

Si $T(t) > T_{\text{critique}}$, une alerte sera générée et la fréquence des capteurs sera augmentée.

3.1.6 Modélisation des Variations Rapides

Outre les tests statistiques classiques, des dérivées premières des séries temporelles sont calculées pour détecter les changements rapides dans les conditions météorologiques. Par exemple, la dérivée première de la température par rapport au temps, notée $\frac{dT}{dt}$, peut indiquer un changement brusque de température, tel qu'une tempête. Si

$$\frac{dT}{dt} > \Delta T_{\text{critique}},$$

alors l'ajustement de la fréquence d'échantillonnage est déclenché.

3.1.7 Implémentation Technique de l'Adaptation

L'implémentation de la modification de la fréquence des capteurs peut être réalisée par une fonction qui ajuste l'intervalle entre deux acquisitions en fonction des événements détectés. Le processus est structuré comme ceci :

Détection de l'Événement (ex. Canicule) à l'aide du seuil

Un algorithme analyse les données en temps réel et détecte les événements de dépassement de seuil, comme une température supérieure à 35°C. Pour ce faire, une fonction de détection pourrait être implémentée comme ceci :

```
# Définition des topics MQTT
MQTT_TOPIC_SENSOR = "sensor/acquisition" # Topic pour acquisition de données
MQTT_TOPIC_ALERT = "sensor/ajustement"   # Topic pour les alertes et ajustement fréquence

# Création d'un client MQTT
client = mqtt.Client()

# Abonnement du client au topic d'acquisition pour recevoir les messages
client.subscribe(MQTT_TOPIC_SENSOR)

# Fonction appelée lorsqu'un message est reçu sur un topic auquel le client est abonné
def on_message(client, userdata, msg):
    # Conversion du message JSON reçu en dictionnaire Python
    data = json.loads(msg.payload)
    temperature = data["temperature"] # Extraction de la température du dictionnaire

    # Vérification si la température dépasse le seuil défini
    if temperature > 35:
        # Création d'un dictionnaire pour l'alerte
        alert = {"event": "heatwave", "value": temperature}
        # Publication de l'alerte sur le topic d'alerte
        client.publish(MQTT_TOPIC_ALERT, json.dumps(alert))
```

Le premier topic, MQTT-TOPIC-SENSOR, est utilisé pour recevoir les données de température envoyées par le capteur. Le second topic, MQTT-TOPIC-ALERT, est destiné à publier des alertes de canicule si la température dépasse un seuil critique et donc ajuster la fréquence.

Une instance du client MQTT est ensuite créée avec `client = mqtt.Client()`, ce qui permet d'initialiser une connexion avec un broker MQTT pour gérer la communication. Une fonction appelée `onMessage` est définie pour être le gestionnaire d'événements, appelée chaque fois qu'un message est reçu sur un topic auquel le client est abonné. Dans cette fonction, le message reçu, formaté en JSON, est décodé pour être converti en dictionnaire Python. La température est extraite de ce dictionnaire pour être utilisée dans les vérifications suivantes.

La fonction procède alors à une vérification de la température en comparant sa valeur à un seuil prédéfini, ici, 35 degrés. Si la température dépasse ce seuil, cela indique un risque de canicule. Dans ce cas, un dictionnaire d'alerte est créé, contenant des informations sur l'événement et la température mesurée. Cette alerte est ensuite publiée sur le topic d'alerte en étant convertie en format JSON.

Enfin, le client se connecte au broker MQTT local avec `client.connect("localhost")`, ce qui lui permet de commencer à communiquer avec le système. Pour recevoir les messages de température, le client s'abonne au topic correspondant.

Ajustement de l'Intervalle d'Échantillonnage

Un client MQTT s'abonne au topic d'ajustement pour modifier dynamiquement l'intervalle d'échantillonnage des capteurs en fonction des événements détectés. Lorsqu'une alerte est publiée, la fréquence d'acquisition des données est mise à jour automatiquement. Une telle fonction peut être implémentée comme ceci :

```
# Fonction appelée lorsqu'un message est reçu sur le topic d'ajustement
def on_alert_message(client, userdata, msg):
    """
    Cette fonction est déclenchée lorsqu'un message est reçu sur le topic
    d'ajustement. Elle adapte la fréquence d'échantillonnage des capteurs
    en fonction de l'événement détecté.
    """
    client.subscribe(MQTT_TOPIC_ALERT)

    global sampling_interval # Variable globale pour stocker l'intervalle d'échantillonnage

    # Décodage du message JSON en dictionnaire Python
    alert_data = json.loads(msg.payload)
    event = alert_data.get("event", "") # Récupération de l'événement, valeur par défaut vide

    # Vérification du type d'événement reçu et ajustement de la fréquence
    if event == "heatwave":
        sampling_interval = 30 # Réduction de l'intervalle à 30 secondes en cas de canicule
    else:
        sampling_interval = 600 # Retour à l'intervalle normal de 10 minutes
```

La fonction `on_alert_message` est abonnée au topic MQTT dédié aux ajustements (`sensor/ajustement`). Cela signifie qu'à chaque fois qu'un message est publié sur ce topic, cette fonction est automatiquement appelée pour traiter l'information reçue.

Dans un système MQTT, un client peut s'abonner à un ou plusieurs topics pour recevoir des messages envoyés par d'autres clients. Ici, `on_alert_message` est associée au topic d'ajustement, ce qui lui permet de réagir immédiatement lorsqu'un événement critique, comme une vague de chaleur ("`heatwave`"), est détecté et signalé via une alerte.

Ainsi, dès qu'un message est publié sur `sensor/ajustement`, la fonction `on_alert_message` est déclenchée et ajuste la fréquence d'échantillonnage en conséquence.

3.1.8 Intégration d'un modèle de régression multiple pour la prédiction de la fréquence

Pour optimiser la fréquence d'échantillonnage en fonction des conditions environnementales, nous allons aussi intégrer un modèle d'apprentissage machine sous la forme de régression multiple [4]. Une régression est une méthode statistique qui permet de modéliser la relation entre une variable cible (ici, la fréquence d'échantillonnage) et une ou plusieurs variables explicatives (comme la température, l'humidité et la pression). L'objectif est de comprendre comment ces variables interagissent et influencent la variable cible, afin de faire des prédictions ou des ajustements précis.

L'intérêt de faire une régression réside dans sa capacité à **capturer des relations complexes** entre les variables. Contrairement à des méthodes simples basées sur des seuils fixes ou des tests statistiques, une régression permet d'adopter une **logique prédictive dynamique**. Par exemple, si la température augmente, le modèle peut prédire comment cela affectera la fréquence d'échantillonnage et ajuster celle-ci en conséquence. Cela rend le système plus intelligent et adaptatif, car il tient compte des tendances et des interactions entre plusieurs facteurs.

La régression multiple repose sur l'hypothèse que la variable cible y peut être exprimée comme une combinaison linéaire des variables explicatives $x_1, x_2, \dots, x_n$, selon l'équation suivante :

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

où β_0 est l'ordonnée à l'origine, β_i sont les coefficients de régression associés à chaque variable explicative, et ϵ représente un terme d'erreur aléatoire. L'objectif est d'estimer ces coefficients pour minimiser l'erreur entre les prédictions $\hat{y}$ et les valeurs réelles y .

L'estimation des coefficients se fait par la méthode des **moindres carrés ordinaires (OLS)**, qui minimise la somme des carrés des erreurs résiduelles :

$$\min_{\beta} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Le modèle est entraîné sur des données historiques, divisées en un jeu d'entraînement (70-80%) et un jeu de test (20-30%). L'entraînement vise à minimiser l'erreur quadratique moyenne (MSE) :

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

D'autres métriques, comme le coefficient de détermination R^2 [5] et l'erreur absolue moyenne (MAE) [3], sont utilisées pour évaluer la qualité du modèle.

Pour éviter le surajustement, une validation croisée en k -fold est appliquée. Cette technique consiste à diviser l'ensemble des données en k sous-ensembles (ou "folds") de taille égale. Le modèle est ensuite entraîné sur $k - 1$ de ces sous-ensembles, tandis que le sous-ensemble restant est utilisé pour tester le modèle. Ce processus est répété k fois, chaque sous-ensemble étant utilisé une fois comme ensemble de test. À la fin de cette procédure, les performances du modèle sont évaluées en calculant la moyenne des scores obtenus sur chaque fold.

La validation croisée en k -fold permet de s'assurer que le modèle généralise bien et n'est pas trop spécifique aux données d'entraînement, réduisant ainsi le risque de surajustement.

Une fois validé, le modèle ajuste la fréquence d'échantillonnage en temps réel. Si une augmentation soudaine de température ou d'humidité est prédite, il recommande d'augmenter la fréquence pour capturer des données plus détaillées. Inversement, en conditions stables, il réduit la fréquence pour économiser de l'énergie sans perte significative d'information.

3.2 Implémentation d'un proxy au sein de l'architecture

L'intégration d'un proxy permet d'optimiser la gestion des ressources en orchestrant intelligemment la distribution des messages entre le cluster et le VPS en fonction de la charge et des besoins. En s'appuyant

sur une architecture modulaire et flexible, le proxy facilite l'acheminement des messages MQTT (section 2.2.2) et renforce la résilience globale du dispositif.

Cette partie présente l'architecture du proxy, ses principales fonctions ainsi que les mécanismes de routage des messages. Nous détaillerons le rôle des différents éléments qui le composent : les routeurs, les fonctions et les brokers MQTT (section 2.2.2) et expliquerons les stratégies mises en place pour garantir une communication fiable et efficace.

3.2.1 Présentation Générale du Proxy

Le schéma de la figure 6 illustre l'architecture mise en place pour l'implémentation du proxy, une explication de l'exécution de l'architecture sur cette figure est présente dans la section 3.2.2. Cette architecture est structurée en trois composantes principales : les routeurs, les fonctions et les brokers MQTT.

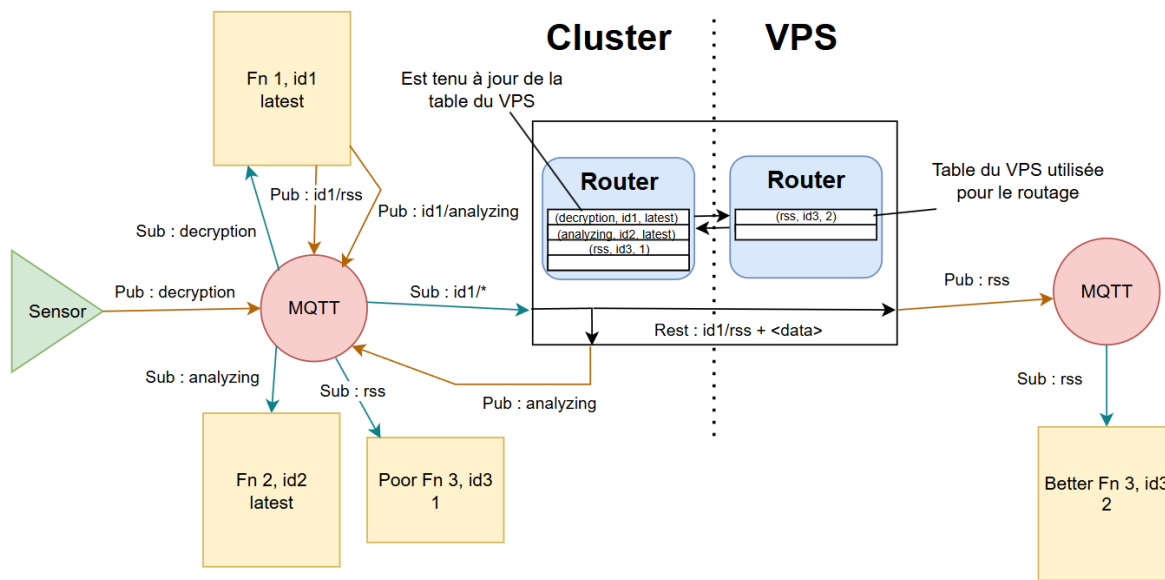


Figure 6: Architecture du proxy

Fonctions

Les fonctions peuvent s'exécuter soit sur le cluster, soit sur le VPS, en fonction de la charge du cluster. Elles peuvent être abonnées à plusieurs topics (section 2.2.2) et elles sont exécutées lorsqu'un message est reçu sur l'un de ces topics. Les fonctions peuvent publier, à leur tour, des messages sur d'autres topics pendant leur exécution.

Pour qu'une fonction soit exécutée lorsqu'un topic reçoit un message, nous avons besoin d'un MQTT Connector. Cette contrainte est imposée par OpenFaaS, qui ne prend pas en charge nativement les fonctions déclenchées par MQTT. Le MQTT Connector agit comme un client MQTT qui, lorsqu'il reçoit un message sur un topic, le transmet à une fonction spécifique via une requête HTTP POST.

Afin d'assurer la compatibilité avec notre architecture, il est nécessaire de modifier le fichier de configuration et le code de la fonction. En particulier, celle-ci doit publier en utilisant son identifiant de fonction comme préfixe.

Configuration pour le déploiement

```
1 version: 1.0
2 provider:
3   name: openfaas
4   gateway: http://127.0.0.1:8080
5 functions:
6   chaleur:
7     lang: golang-middleware
8     handler: ./chaleur
9     image: myceliumir/chaleur:latest
10    annotations:
11      topic: fonction_chaleur
12    environment:
13      FUNCTION_ID: chaleur
14
```

Figure 7: Modification du fichier de configuration

On peut voir sur la figure 7 les modifications à appliquer sur le fichier de configuration, il faut ajouter une variable d'environnement.

Modification du code de la fonction

```
1 func SendMQTT(messageJSON []byte, topic string, url string) {
2     functionID := os.Getenv("FUNCTION_ID")
3     fullTopic := fmt.Sprintf("%s/%s", functionID, topic)
4
5     mqttOpts := mqtt.NewClientOptions()
6     mqttOpts.AddBroker(url)
7     mqttOpts.SetClientID("fonction_chaleur")
8
9     mqttClient := mqtt.NewClient(mqttOpts)
10    if mqttToken := mqttClient.Connect(); mqttToken.Wait() && mqttToken.Error() != nil {
11        log.Fatal(mqttToken.Error())
12    }
13
14    mqttToken := mqttClient.Publish(fullTopic, 0, false, string(messageJSON))
15    mqttToken.Wait()
16
17    fmt.Printf("Message publie sur le topic %s: %s\n", fullTopic, messageJSON)
18
19    mqttClient.Disconnect(250)
20 }
21
```

Figure 8: Modifications appliquées aux fonctions

Dans le code lié à notre fonction, il faut modifier la fonction d'envoi de message MQTT, comme sur la figure 8 pour récupérer l'id de fonction et le placer en préfixe du topic envoyé.

Routeurs

L'architecture repose sur deux routeurs, l'un déployé sur le cluster et l'autre sur le VPS conformément à la figure 6. Ces routeurs échangent régulièrement des signaux de vérification (pings) afin de s'assurer du bon fonctionnement de la connexion. Chaque routeur maintient une table de correspondances qu'il transmet dès qu'un changement sur la table arrive (nouvelle fonction, scale up, etc...) à l'autre routeur contenant les informations suivantes :

- Les topics auxquels les fonctions du cluster ou du VPS sont abonnées.
- L'identifiant unique qui permet d'identifier une fonction.
- Le tag de la fonction, un nombre, le plus haut représente la plus haute priorité, s'il existe une seule version de cette fonction, alors le tag affiché sera 0.

Lorsqu'un message MQTT est reçu par l'un des routeurs, celui-ci applique la politique de routage définie dans la section 3.2.4.

Pour actualiser sa table de correspondance, le routeur regarde régulièrement les fonctions présentes pour en récupérer leurs identifiants et leurs tags ainsi que les annotations qui indiquent à quels topics ces fonctions sont abonnées. Si le tag correspond à 0 c'est qu'il existe une seule version de cette fonction sinon, les tags correspondront à des numéros pour la priorité.

Brokers MQTT

Les brokers MQTT sont responsables de la distribution des messages reçus par les routeurs vers les fonctions correspondantes.

3.2.2 Exemple d'Exécution du Proxy

Nous allons nous placer dans le cadre de l'exemple de la figure 6 pour illustrer au mieux le fonctionnement du proxy :

1. **Un capteur envoie une donnée** : Un capteur publie ses données sur le topic *decryption* sur le broker du cluster.
2. **Exécution de la fonction 1** : La fonction 1 s'exécute et publie donc par la suite sur un topic hiérarchique commençant par son id, propre à elle donc, ses données sur deux topics : *id1/analyzing* et *id1/rss*.
3. **Transmission au routeur** : Le broker transmet le message au routeur du cluster comme celui-ci est abonné à toutes les hiérarchies des fonctions, sur la figure 6, "*id1/**".
4. **Routage des messages MQTT** : Conformément à la politique de routage décrite section 3.2.4, le routeur redirige le message du topic *analyzing* sur le broker du cluster, et transmet le message du topic *rss* sur le routeur du VPS via REST puis sur le broker du VPS.
5. **Déclenchement et exécution de Fn2 et de Better Fn3** : Better Fn3 est accessible et a une plus haute priorité que Poor Fn3, c'est donc la fonction Better Fn3 qui est exécutée
6. **Bouclage du processus** : Le cycle peut se répéter à partir de l'étape 2, chaque fonction publiera sur sa hiérarchie.

3.2.3 Monitoring du Cluster

Pour assurer un routage dynamique efficace, les routeurs doivent disposer d'informations en temps réel sur l'état de charge du cluster afin d'optimiser la répartition du trafic. Pour cela, une fonctionnalité intégrée aux routeurs récupérera en continu les métriques de charge CPU et d'utilisation de la mémoire du cluster via l'API exposée par Kube Metrics Server [12]. Ces données permettront d'ajuster dynamiquement les décisions de routage. À terme, des règles internes pourront être définies au sein du système de monitoring afin de déléster certaines fonctions vers le VPS en cas de surcharge du cluster, tout en réduisant significativement le trafic qui lui est envoyé.

3.2.4 Politique de Routage des Messages MQTT

Le routage des messages MQTT est déterminé selon cinq cas distincts :

- **Aucune fonction ne contient le topic sur lequel le message a été publié dans l'un de ses tags** : Un message d'erreur est retourné, on ne sait pas quelle fonction abonnée à ce topic doit être lancée.

- **Seul le VPS ou le cluster contient une seule fonction avec le topic du message** : Le message est transmis à la fonction concernée.
- **Plusieurs fonctions avec des identifiants distincts sont abonnées au topic du message** : Le message est envoyé à toutes les fonctions concernées.
- **Plusieurs fonctions partagent le même topic, le même identifiant et le même tag** :
 - Si ces fonctions sont déployées à la fois sur le cluster et sur le VPS, le monitoring (section 3.2.3) du routeur est utilisé pour déterminer vers quel broker rediriger le message.
 - Si plusieurs de ces fonctions sont présentes sur le broker qui a reçu le message, alors OpenFaaS applique son mécanisme de répartition de charge pour distribuer le message entre elles.
- **Plusieurs fonctions sont abonnées au topic du message, ont le même identifiant mais un tag différent** : On redirige le message vers la fonction ayant le tag de priorité le plus haut.

3.2.5 Avantages de l'Architecture

- **Redondance et résilience** : Les routeurs effectuent des pings réguliers. En cas de déconnexion, ils fonctionnent indépendamment, puis reprennent leur fonctionnement standard une fois la connexion rétablie.
- **Transparence pour les développeurs** : Les fonctions s'exécutent de manière identique sur le cluster et le VPS.
- **Gestion des priorités** : Si plusieurs fonctions ayant le même identifiant existent, mais avec des tags de priorités différents, le message est dirigé vers la fonction ayant la priorité la plus élevée.

3.3 Intégration du scénario inondation

L'intégration d'un scénario d'inondation à partir d'une application de suivi environnemental développée par le groupe d'étude pratique de 3ème année vient enrichir Mycélium 4.0, permettant d'émettre des prédictions dans le but d'alerter l'utilisateur en cas d'inondation. L'ajout de ce nouveau scénario a cependant nécessité des adaptations pour fonctionner au sein de Mycélium, qui vont être détaillées ci-après.

3.3.1 Adaptation à l'architecture Mycélium

La communication des données récoltées par les capteurs pour ce scénario sont transmises via le protocole MQTT, au même titre que tous les scénarios du projet, mais cela doit être adapté puisque que c'était initialement supporté par NATS [14], un système de messagerie broker plus utilisé dans le cloud. Les remontées d'alertes à l'utilisateur se font ensuite par le flux RSS existant au lieu des notifications discord précédemment. Ces adaptations de la gestion de données de ce scénario à l'environnement de travail virtuel (VM) permet donc de valider l'intégration du scénario inondation à l'architecture en place. Du côté du fonctionnement sur le cluster Raspberry Pi, le scénario inondation fonctionne sur une architecture légère basée sur K3S qui permet de ne pas surexploiter les ressources du cluster.

3.3.2 Ajustement du modèle prédictif

Le modèle prédictif utilisé pour les prédictions d'inondation fonctionne selon des réseaux de neurones récurrents Long Short Term Memory (LSTM) [15]. Ce type de réseau de neurones en particulier capture efficacement les dépendances à long terme dans les séquences, le rendant donc parfaitement adéquat pour les prédictions sur les données météorologiques qui sont des séries temporelles complexes. L'objectif est donc de pouvoir anticiper des événements climatiques critiques (en l'occurrence des inondations grâce une analyse par le modèle à partir des différentes données collectées par les capteurs).

Mais le modèle avait d’abord été entraîné avec des données météorologiques et hydrologiques australiennes qui diffèrent donc tout à fait du climat local : il a donc été nécessaire de réentraîner le modèle avec les données relevées par les capteurs de l’OSUR.

Suite à cet entraînement, les prédictions vont pouvoir être mises en place à partir des flux continus de données collectées en temps réel d’une part par les capteurs de l’OSUR et d’autre part par nos capteurs du réseau LoRaWAN, données qui seront également

Grâce à ce ré-entraînement du modèle, nous pouvons garantir la pertinence des prédictions et ainsi des alertes reçues par l’utilisateur.

4 Conclusion

Ce rapport de conception nous permet de visualiser la mise en œuvre des spécifications détaillées dans le rapport précédent. Nous y avons visualisé de façon plus concrète et pointue le fonctionnement de Mycélium 4.0 et la procédure de traitement des données, de leur collecte au niveau des capteurs jusqu’à la notification de l’utilisateur, en passant par le traitement des données.

Mycélium 4.0 se basant largement sur la conception existante de Mycélium 3.0, c’est celle-ci qui est présentée dans un premier temps. L’articulation des nœuds Solo, du cluster et du VPS, les 3 éléments centraux de l’architecture, dans l’acheminement des données y est alors exposée. C’est ce qui sert de base à Mycélium 4.0 qui va en effet utiliser une approche modulaire de la gestion des données.

Dans un second temps, nous détaillons l’architecture de Mycélium 4.0, il ressort que les changements logiciels effectués permettent une optimisation de la gestion et l’acheminement des données ainsi que l’exécution des fonctions. L’introduction d’un proxy qui route les messages entre le Cluster et le VPS permet d’optimiser les transferts des données entre ces derniers et d’utiliser l’endroit le plus adapté pour l’exécution de chaque fonction. Cet ajout permet alors de se prémunir contre une perte de connexion mais aussi de transférer des calculs sur le VPS lorsque le Cluster est saturé. La performance du stockage de données est assurée par l’utilisation de InfluxDB et Kubernetes facilite le déploiement des fonctions OpenFaas. L’intégration du modèle de régression multiple permet d’optimiser la fréquence d’échantillonnage améliore le système, le rendant plus adaptatif grâce aux prédictions, induisant ainsi une meilleure réactivité du dispositif, tout en permettant une optimisation de la consommation énergétique. L’intégration du scénario inondation vient enrichir Mycélium par l’adaptation d’un projet parallèle à l’environnement.

L’architecture que nous avons conçue pour Mycélium 4.0 est plus stable et robuste. Elle permet donc d’assurer une meilleure gestion des données et une facilité d’exploitation, deux éléments qui sont en l’occurrence le point d’orgue du projet. Cette nouvelle architecture nous rapproche de l’atteinte des objectifs : l’exploitation des données par divers utilisateurs qui pourront être acteurs des enjeux socio-environnementaux dans lesquels se place Mycélium, avec une prise en main abordable pour les utilisateurs d’une problématique de distribution de calculs complexes.

References

- [1] INRIA, Test de Kolmogorov-Smirnov. Disponible sur : <https://mistis.inrialpes.fr/software/SMEL/cours/ts/node7.html>.
- [2] Wikipedia contributors, Levene's test. Disponible sur : https://en.wikipedia.org/wiki/Levene%27s_test.
- [3] Qu'est-ce que c'est : l'erreur absolue moyenne. Available at : <https://fr.statisticseasily.com/glossario/what-is-mean-absolute-error-mae/>.
- [4] Laffly, Dominique. *Régression multiple : principes et exemples d'application*. Available at: https://web-new.univ-pau.fr/RECHERCHE/SET/LAFLY/docs_laffly/Laffly_regression%20multiple.pdf.
- [5] Ionos. R carré en régression. Disponible sur : <https://www.ionos.fr/digitalguide/sites-internet/developpement-web/r-carre-en-r/#:~:text=R%20carr%20est%20une%20mesure,qualit%20des%20modles%20de%20rgression>.
- [6] Documentation officielle de MQTT. Disponible sur : <https://mqtt.org/>.
- [7] Définition d'un Topic MQTT par StackHero. Disponible sur : <https://www.stackhero.io/fr-fr/services/Mosquitto/documentations/Pour-commencer/Que-sont-les-topics-dans-MQTT>.
- [8] Documentation officielle de Mosquitto. Disponible sur : <https://mosquitto.org/>.
- [9] *Serverless Architecture with OpenFaaS and Java* par Xenonstack. Disponible sur : <https://www.xenonstack.com/blog/serverless-open-faas-java>.
- [10] *Le serverless ou informatique sans serveur, qu'est-ce que c'est ?* par Redhat. Disponible sur : <https://www.redhat.com/fr/topics/cloud-native-apps/what-is-serverless>.
- [11] Documentation officielle de Kubernetes. Disponible sur : <https://kubernetes.io/fr/docs/home/>
- [12] Github officiel de Kube Metrics Server. Disponible sur : <https://github.com/kubernetes-sigs/metrics-server>
- [13] Présentation du boîtier Noeud SoLo sur site officiel. Disponible sur : <https://www.connecsens.org/nos-technologies/noeud-solo>
- [14] Documentation officielle de NATS. Disponible sur : <https://nats.io/>
- [15] Long Short Term Memory (LSTM) : de quoi s'agit-il ? Disponible sur : <https://datascientest.com/long-short-term-memory-tout-savoir>
- [16] Page Wikipedia Fog Computing. Disponible sur : https://en.wikipedia.org/wiki/Fog_computing
- [17] Page Wikipedia Cloud Computing. Disponible sur : https://fr.wikipedia.org/wiki/Cloud_computing
- [18] Documentation officielle de Grafana. Disponible sur : <https://grafana.com/docs/grafana/latest/>
- [19] Que sont les flux RSS ? Disponible sur : <https://support.microsoft.com/fr-fr/office/que-sont-les-flux-rss-e8aaebc3-a0a7-40cd-9e10-88f9c1e74b97>
- [20] Site officiel de Raspberry Pi. Disponible sur : <https://www.raspberrypi.com/>
- [21] Documentation officielle de K3S. Disponible sur : <https://docs.k3s.io/>

- [22] Article sur l'organisation FaaS. Disponible sur : [https://www.ibm.com/fr-fr/think/topics/faas#:~:text=Le%20FaaS%20\(fonction%20%C3%A0%20la%20demande\)%20est%20un%20service%20de,lancement%20d'applications%20de%20microservices.](https://www.ibm.com/fr-fr/think/topics/faas#:~:text=Le%20FaaS%20(fonction%20%C3%A0%20la%20demande)%20est%20un%20service%20de,lancement%20d'applications%20de%20microservices.)
- [23] Article sur les event Broker. Disponible sur : <https://solace.com/what-is-an-event-broker/>
- [24] Site Officiel de ChirpStack. Disponible sur : <https://www.chirpstack.io/>
- [25] Site Officiel de QEMU. Disponible sur : <https://www.qemu.org/>